

■ 2.6.18 Special Topic: Running External Programs

<code>!command</code>	“shell escape”: take a complete line of <i>Mathematica</i> input, and run it as an external command
<code>Run["command", arg₁, ...]</code>	run an external command from within <i>Mathematica</i>

Running external commands.

By starting a line with `!`, you can effectively direct the input you type not to *Mathematica*, but rather to your operating system. Anything after `!` that you type on the line is taken as an external command to execute. Any output that the command generates is immediately displayed.

You should know that some *Mathematica* front ends cannot support this kind of interaction. They will usually give you other ways to execute external commands.

When you put an <code>!</code> at the beginning of a line, the rest of the line is interpreted as an operating system command. The UNIX command <code>date</code> used in this example returns the current date and time.	<pre>In[1]:= !date Sun Apr 24 02:59:00 CDT 1988</pre>
---	---

The way *Mathematica* uses `!command` is typical of the way “shell escapes” work in programs running under the UNIX operating system. You will usually be able to start an interactive shell just by typing `!` on its own.

`!` allows you to execute external programs when you type input to *Mathematica*. You can also execute external programs from within a *Mathematica* program, using `Run`.

With `Run`, however, you can neither give input to the external program, nor receive output from it. (Under the UNIX operating system, both standard input and output for the external program are taken to be `/dev/null`.)

This executes the command <code>date</code> , but shows no output from it. The returned value is an “exit code” from the operating system.	<pre>In[1]:= Run["date"] Sun Apr 24 02:59:00 CDT 1988 Out[1]= 0</pre>
--	---

You can use `Run` to execute a command that has some external effect. For example, you might use `Run` to print graphics data that you are building up in a particular file.

You can generate the commands that you want to use in `Run` from within *Mathe-*

matica. All that **Run** does is to take the textual forms of any arguments you specify, and then join them together, with spaces in between, and using the resulting text as the command to execute.

Under most operating systems, you can specify a command, then give some “arguments” for it, separated by spaces. In this case, the first argument to **Run** gives the name of the “command”; subsequent arguments give the “arguments” of the command.

One way to execute an external program on data from *Mathematica* is first to put the data in a file, and then call the external program using **Run**, with the name of the file given as an appropriate argument. A much easier way to do this, however, is to make use of pipes.

<code>expr >> "!command"</code>	run <i>command</i> , with the textual form of <i>expr</i> as input
<code><<"!command"</code>	run <i>command</i> , and take its output as <i>Mathematica</i> input
<code>RunThrough["command", expr]</code>	run <i>command</i> , using <i>expr</i> as input, and reading the output back into <i>Mathematica</i>
<code>OpenTemporary[]</code>	open a temporary file, with a unique name
<code>OpenWrite["!command"]</code>	start executing <i>command</i> , and open a pipe to give input to it
<code>Write["!command", expr]</code>	send an expression to an external command through a pipe
<code>Display["!command", graphics]</code>	send POSTSCRIPT graphics to an external command
<code>Close["!command"]</code>	stop executing <i>command</i> , and close the pipe to it

Running external commands with pipes.

When you use `<<"!command"`, the external command you specify is automatically started. *Mathematica* goes on taking input from the command until the command terminates.

When you use `expr>>"!command"`, the external command is again automatically started. *Mathematica* sends the textual form of *expr* as input to the command, and then terminates the input.

You can use `>>` and `<<` to get data to and from external programs. Both of these operators can transmit data only in one direction. Sometimes you may need

to send data from *Mathematica* to an external program, and then get results back to *Mathematica* from the same program.

The most general way to set up communication of this kind is to put data you generate into temporary files, and then to read and write these files from *Mathematica*. You can use the function `OpenTemporary[ ]` to open a temporary file with a unique name.

A more convenient way to set up two-way communication is to use the function `RunThrough`. This function starts an external command, sends the textual form of a *Mathematica* expression to it, and then takes its output and reads it back into *Mathematica*. Notice that in `RunThrough`, as in `Run`, the text of your external command is not prefaced by `!`.

This feeds the expression 78 to the external program `cat`, which in this case simply echoes the text of the expression. The output from `cat` is then read back into *Mathematica*.

```
In[2]:= RunThrough["cat", 78]
Out[2]= 78
```

Functions like `>>` and `RunThrough` first start an external program, then send a single piece of *Mathematica* output to the program. Sometimes you may want to start an external program, and then send a sequence of expressions to it. You can do this in essentially the same way as you write sequences of expressions to files.

The first step is to call `OpenWrite["!command"]`. This starts up the external program you specify. Then you can send a sequence of expressions to the program by calling `Write["!command", expr]` on each one. You can tell the external program that you have finished sending input to it by calling `Close["!command"]`.

Once you have started an external program using `OpenWrite`, it is important that every time you refer to the program you do so by giving exactly the same string. `OpenWrite` always returns the name of the object it opens. It is usually convenient to assign this name as the value of a variable, and then use the variable when you need to specify the program in `Write` and `Close`.